Block Operators
Milestone 3

Anjali Kanvinde Bhavana Anand Gomathy Dhanya S Sameeksha Bhatia

1. Introduction

Semantic query processing systems use LLMs to perform operations such as filtering, mapping, and joining unstructured data. These queries often involve multiple dependent operations.

Kalypso uses a producer-consumer pipeline, allowing downstream operators to process tuples as soon as upstream results are available. However, its semantic operators issue a separate LLM request for each tuple or join pair. Although vLLM supports continuous batching and prefix caching, individual requests still introduce overhead and repeat prompt content.

Our project explores block-based execution, where multiple tuples are processed in a single LLM request. This reduces request counts but can increase prompt length, output generation time, and the risk of incorrect responses. Batching can also change the workload of downstream operators.

We analyze batching for filter, map, and join operators, both independently and in pipelines, across different workloads and block sizes. Based on these findings, we propose an algorithm to select block sizes for each operator and validate it against static batching strategies and tuple-wise execution.

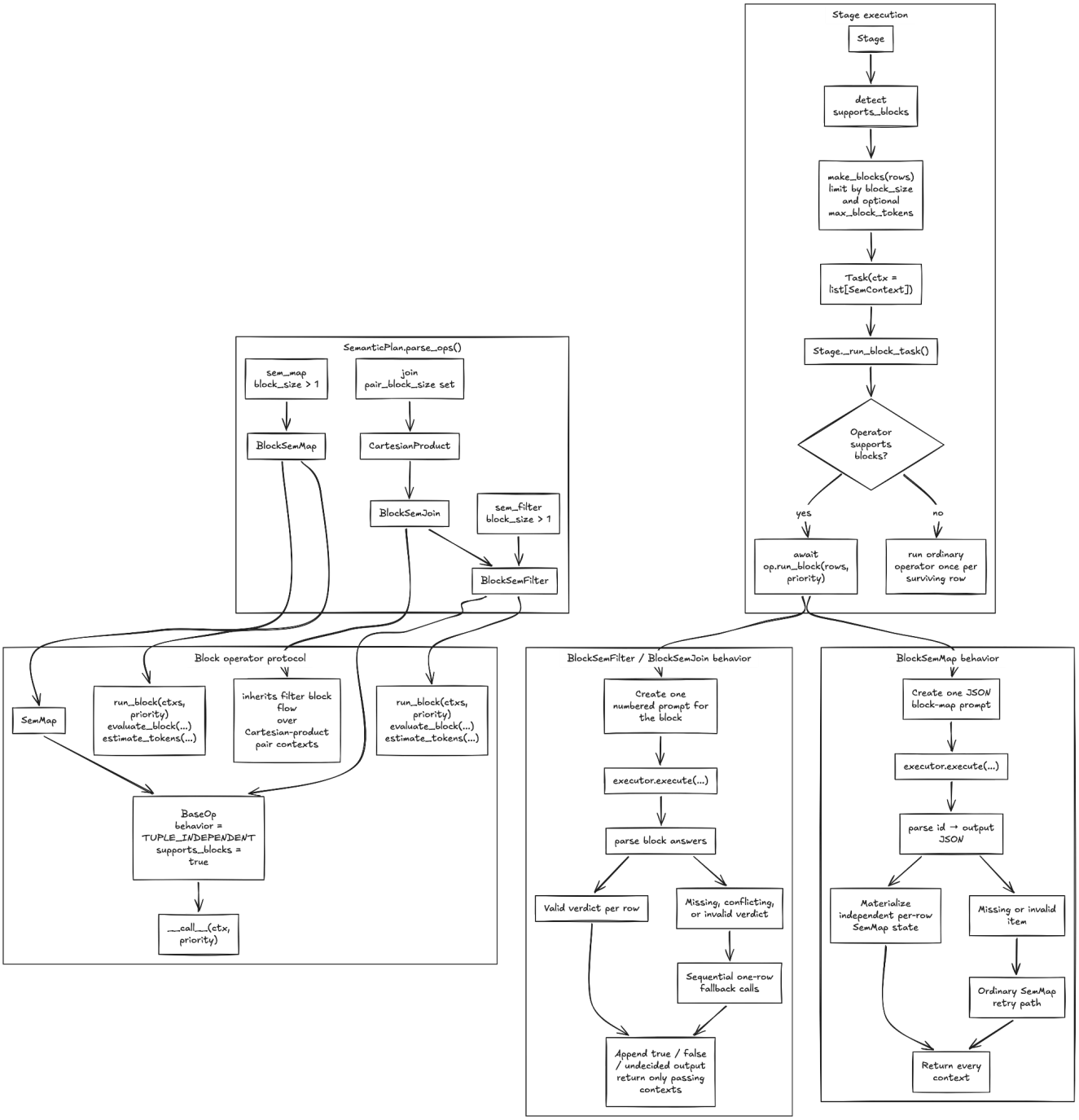
2. Block Semantic Operators

We implemented block-based versions of the three semantic operators: filter, join, and map. Each operator processes multiple tuples or pairs in a single LLM request, sharing instructions across inputs. All three follow a common execution interface but differ in prompt formatting, output parsing, and error handling.

Feature	Block Filter	Block Join	Block Map
Class	BlockSemFilter	CartesianProduct → BlockSemJoin (a BlockSemFilter subclass)	BlockSemMap (a SemMap subclass)
Prompt	Instruction once; rows numbered 1..k, one per line	Same; each item is “Left item: … / Right item: …”	Instruction once; rows as JSON [{id, context}]
Answer format	k lines: <n>: YES|NO	Same as filter	JSON {items: [{id, output}]}
Parsing	Tolerant line parser (YES/NO/TRUE/FALSE); each ID once, in range	Same as filter	Strict JSON; each ID must be in 1..k with one string output
Bad or missing answer	One-row fallback call, run sequentially	Same as filter	Ordinary one-row SemMap path, with its retry

Output budget	16 + 6·k tokens	16 + 6·k tokens	32 + each row's SemMap budget, capped at 512
Result	True / false / undecided; passing rows continue	Pairs retain idx and right_idx	Each row stored as if mapped alone; answer-length estimator updated
Key parameters	block_size, max_tokens_per_row, single_max_tokens, negate	pair_block_size, max_tokens_per_pair	block_size, max_tokens_overhead; pin not allowed

3. Architecture Diagram



4. Hardware, Datasets and Execution Modes

4.1 Hardware and model-serving configuration

Component	Configuration	Implication for batching
GPU	1 × NVIDIA L40S, 48 GB (Modal); fresh server per experiment	One-GPU prefill/decode resources govern both tuple-wise and block runs.

Model	Qwen2.5-7B-Instruct, bf16; temperature 0	Controlled decoding setting; responses can still differ across prompts.
Serving stack	Kalypso + vLLM 0.29.0	The baseline benefits from server-side request batching.
Scheduler limits	max_num_seqs=256 max_num_batched_tokens=2048 max_model_len=32768	Large request sets proceed in waves; prompt size limits feasible k.
Caching	Prefix cache enabled; reset before each trial	Repeated instructions or shared prefixes can be reused by tuple-wise requests.

4.2 Datasets and experimental workloads

Dataset	Input length	Labels	Workloads built from it
SemBench movie reviews	~31 tokens (short)	Sentiment	Filter: “Clearly positive?” (100–1865 rows, 7–92% true). Join: “Opposite / same sentiment?” (16×16–64×64 pairs). Map: “Main reason in one sentence” (100–500 rows). Pipelines: Filter → Join, Filter → Map, Map → Filter, Join → Filter.
ContractNLI (NDAs)	~1500 tokens (long); truncated to ~140, ~260, ~500, and ~940 tokens	Entailment, with evidence spans	Filter: “Does the contract entail the hypothesis?” (200–400 rows). Join: Contract × Hypothesis (24×8, 40×8 pairs). Other: Filter, Join, and Map on contracts truncated to fixed lengths.

4.3 Execution modes and metrics

Each configuration uses one warm-up and three timed repetitions and we report median end-to-end wall-clock latency, with experimental arms interleaved. A speedup of 1.0× means the same latency as tuple-wise execution, above 1.0× is faster and below 1.0× is slower. Call counts include block calls plus fallback or repair calls.

For filters and joins we track F1 changes relative to tuple-wise execution; for maps we report ROUGE-1 similarity between block output and the tuple-wise output (1.0 means identical under this metric). F1 against task labels and similarity to a tuple-wise map are different quality indicators; neither should be interpreted as a universal semantic correctness measure.

5. Measured behavior in isolation and in pipelines

vLLM already batches concurrent requests and caches repeated prompt prefixes. Therefore, block-based execution mainly reduces repeated instructions, formatting tokens, and request overhead. However, larger blocks require the model to generate multiple answers sequentially within a single request, increasing generation time. As a

result, batching is most beneficial for short inputs, where per-request overhead accounts for a larger share of execution time.

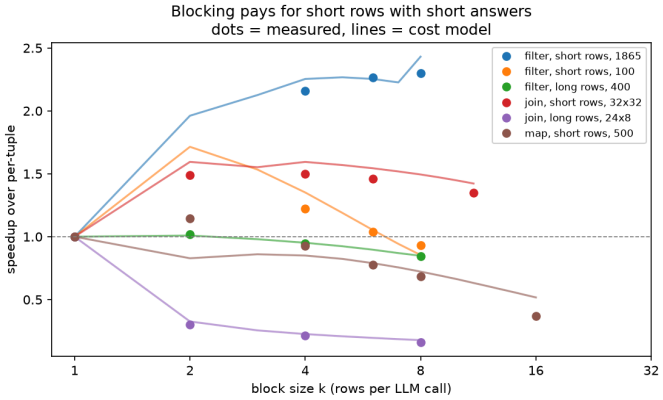
5.1.1 Batching Individual Operators

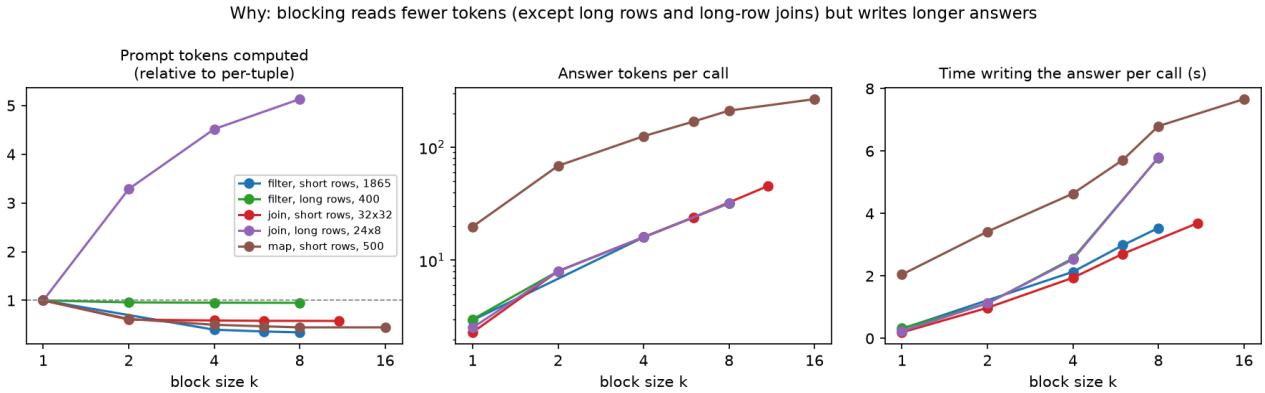
We first evaluate filter, join, and map operators independently across different block sizes and input lengths. We measure execution time, speedup over tuple-wise execution, LLM calls, token usage, and output quality. Quality represents change in F1 relative to tuple-wise execution for filters and joins. For maps, it represents ROUGE-1 similarity to tuple-wise outputs. LLM calls include fallback and repair requests.

Operator / Workload	Block size (k)	Time (s)	Speedup	LLM calls	Tokens/call (input/output)	Quality
Filter — Movie reviews (1,865 rows, 31 tokens)	1	18.32	1.00×	1,865	163 / 3	Baseline
	4	8.50	2.16×	467	216 / 16	+0.10
	6	8.09	2.27×	311	283 / 24	+0.11
	8	7.97	2.30×	234	348 / 32	+0.10
Join — Movie review pairs (32 × 32, 62 tokens/pair)	1	10.43	1.00×	1,024	218 / 2	Baseline
	2	7.00	1.49×	512	230 / 8	−0.05
	4	6.97	1.50×	256	370 / 16	+0.05
	6	7.15	1.46×	171	508 / 24	−0.05
	11	7.73	1.35×	94	855 / 46	−0.28
Map — Movie reviews (500 rows, 31 tokens)	1	7.36	1.00×	500	159 / 20	1.00
	2	6.43	1.14×	250	202 / 69	0.37
	4	7.93	0.93×	125	287 / 125	0.35
	6	9.47	0.78×	87	363 / 169	0.34
	8	10.75	0.69×	65	445 / 211	0.32
	16	19.80	0.37×	51	549 / 267	0.33
Filter — ContractNLI (400 rows, ~1,500 tokens)	1	52.24	1.00×	400	1,636 / 3	Baseline
	2	51.36	1.02×	200	3,085 / 8	−0.13
	4	55.30	0.94×	100	6,079 / 16	−0.08

	8	61.86	0.84×	50	12,066 / 32	−0.19
Join — ContractNLI (24 × 8, ~1,600 tokens/pair)	1	5.58	1.00×	192	1,778 / 3	Baseline
	2	18.41	0.30×	96	3,349 / 8	−0.14
	4	26.30	0.21×	48	6,601 / 16	0.00
	8	34.51	0.16×	24	13,105 / 32	0.00

Observation: Small blocks improve filter and join performance on short inputs, reaching 2.30× and 1.50× speedups, respectively. In contrast, batching long inputs provides little benefit or substantially increases latency. Map batching also performs poorly beyond k = 2 because of longer generated outputs and repair overhead.





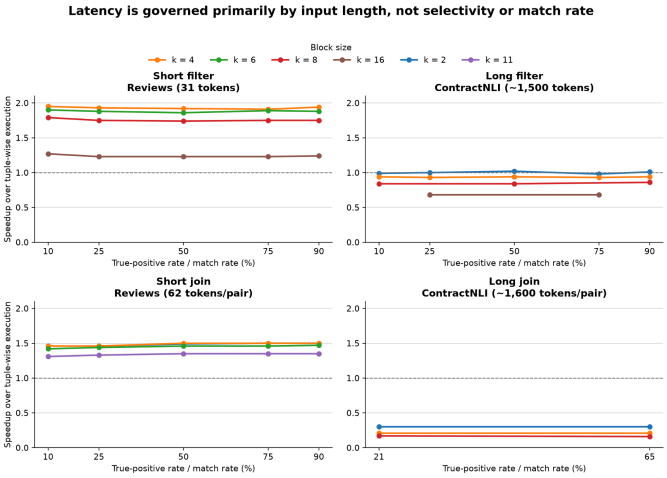
5.1.2 Effect of Selectivity and Match Rate

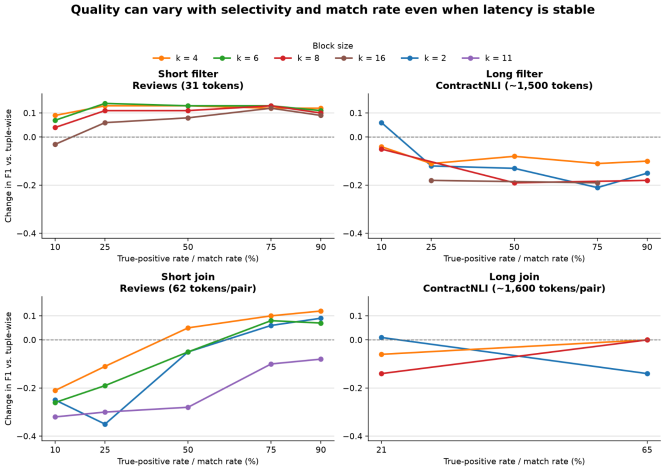
We evaluate whether the proportion of matching tuples affects batching performance. Each entry shows speedup relative to tuple-wise execution, followed by F1 change in parentheses. A dash indicates an untested configuration.

Operator / Dataset	True-positive rate	Per-tuple time (s) / F1	k = 2	k = 4	k = 6 / 8	Larger blocks

Filter — Reviews (31 tokens)	10%	3.95 / 0.70	—	1.95× (+0.09)	1.90× / 1.79× (+0.07 / +0.04)	k16: 1.27× (−0.03)
	25%	3.76 / 0.73	—	1.93× (+0.13)	1.88× / 1.75× (+0.14 / +0.11)	k16: 1.23× (+0.06)
	50%	3.75 / 0.74	—	1.92× (+0.13)	1.86× / 1.74× (+0.13 / +0.11)	k16: 1.23× (+0.08)
	75%	3.81 / 0.75	—	1.91× (+0.12)	1.89× / 1.75× (+0.13 / +0.13)	k16: 1.23× (+0.12)
	90%	3.75 / 0.78	—	1.94× (+0.12)	1.88× / 1.75× (+0.11 / +0.10)	k16: 1.24× (+0.09)
Filter — ContractNLI (~1,500 tokens)	10%	51.21 / 0.26	0.99× (+0.06)	0.94× (−0.04)	k8: 0.84× (−0.05)	—
	25%	24.03 / 0.53	1.00× (−0.12)	0.93× (−0.11)	—	k16: 0.68× (−0.18)
	50%	52.24 / 0.68	1.02× (−0.13)	0.94× (−0.08)	k8: 0.84× (−0.19)	—
	75%	39.73 / 0.78	0.98× (−0.21)	0.93× (−0.11)	—	k16: 0.68× (−0.19)
	90%	53.04 / 0.80	1.01× (−0.15)	0.94× (−0.10)	k8: 0.86× (−0.18)	—
Join — Reviews (62 tokens/pair)	10%	11.05 / 0.50	1.46× (−0.25)	1.46× (−0.21)	k6: 1.42× (−0.26)	k11: 1.31× (−0.32)
	25%	10.83 / 0.71	1.46× (−0.35)	1.46× (−0.11)	k6: 1.44× (−0.19)	k11: 1.33× (−0.30)
	50%	10.43 / 0.78	1.49× (−0.05)	1.50× (+0.05)	k6: 1.46× (−0.05)	k11: 1.35× (−0.28)
	75%	10.74 / 0.80	1.50× (+0.06)	1.50× (+0.10)	k6: 1.46× (+0.08)	k11: 1.35× (−0.10)
	90%	10.54 / 0.81	1.50× (+0.09)	1.50× (+0.12)	k6: 1.47× (+0.07)	k11: 1.35× (−0.08)
Join — ContractNLI (~1,600 tokens/pair)	21%	5.65 / 0.37	0.30× (+0.01)	0.21× (−0.06)	k8: 0.17× (−0.14)	—
	65%	5.58 / 0.57	0.30× (−0.14)	0.21× (0.00)	k8: 0.16× (0.00)	—

Observation: Selectivity and match rate have limited impact on execution speedup. Short-input filters maintain approximately 1.9× speedup at k = 4, while short-input joins achieve approximately 1.5× at k = 2–4. However, match rate affects F1, particularly for joins with low match rates.



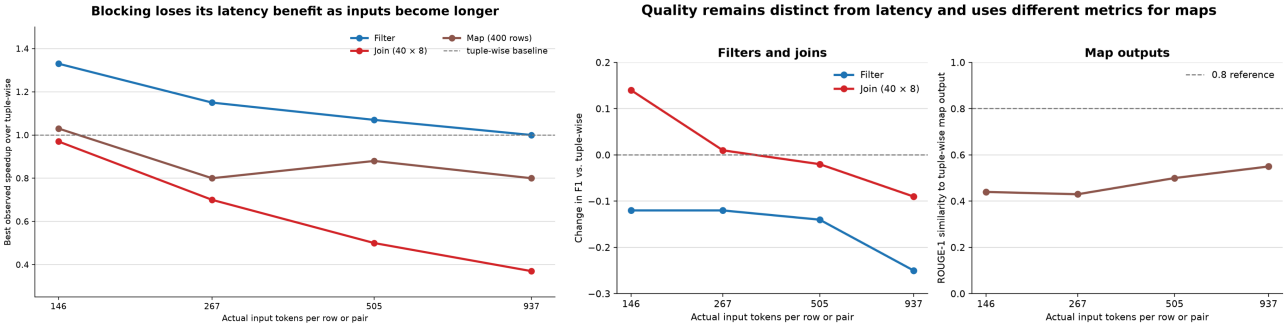


5.1.3 Effect of Input Length

To isolate the effect of input length, we truncate ContractNLI inputs to different token lengths and evaluate each operator over multiple block sizes. The fastest block is selected among k ≥ 2. A speedup below 1.0× indicates that tuple-wise execution is faster. Quality is F1 change for filters and joins, and ROUGE-1 similarity for maps.

Operator	Target / actual tokens	Block sizes tested	Fastest block (k)	Per-tuple time (s)	Best block time (s)	Speedup	Quality
Filter	128 / 146	2, 3, 4, 8, 16, 32	3	7.70	5.81	1.33×	−0.12
	256 / 267	2, 3, 4, 8, 16, 32	3	11.39	9.90	1.15×	−0.12
	512 / 505	2, 3, 4, 8, 16, 32	3	20.09	18.78	1.07×	−0.14
	1024 / 937	2, 3, 4, 8, 16, 32	2	32.98	32.97	1.00×	−0.25
Join (40 × 8)	128 / 139	2, 3, 4, 5, 8, 16, 32	2	3.48	3.57	0.97×	+0.14
	256 / 261	2, 4, 5, 8, 16, 32	2	4.08	5.83	0.70×	+0.01
	512 / 501	2, 4, 5, 8, 16, 32	2	4.90	9.79	0.50×	−0.02
	1024 / 942	2, 4, 8	2	6.47	17.29	0.37×	−0.09
Map (400 rows)	128 / 125	2, 4, 8	2	10.10	9.79	1.03×	0.44
	256 / 244	2, 4, 8	4	13.06	16.23	0.80×	0.43
	512 / 485	2, 4, 8	2	23.13	26.41	0.88×	0.50
	1024 / 916	2, 4, 8	2	41.90	52.67	0.80×	0.55

Observation: Batching becomes less effective as input length increases. Filter speedup falls from 1.33× at 146 tokens to approximately 1.00× at 937 tokens. Join batching is already slower at 139 tokens and deteriorates further with longer inputs. Map batching provides minimal benefit even for short inputs.



These findings motivate a pipeline-aware latency model that considers operator type, input and output lengths, expected downstream cardinality, retries, and pipeline interactions. Quality is reported separately and can be used as an optional guard.

5.2 Batching in Pipelines

We evaluate batching across four two-stage pipelines: Filter → Join, Filter → Map, Map → Filter, and Join → Filter. Each stage can use a different block size, represented as (k_1, k_2), where $(1, 1)$ denotes tuple-wise execution.

We compare execution time, LLM calls, and F1 to determine how batching decisions affect overall pipeline performance. The table reports the fastest measured configuration among the tested block-size combinations, including the tuple-wise baseline. LLM calls include fallback and repair requests. For Join → Filter, the tuple-wise join identified only 0–8% of true pairs. The F1 improvements therefore partly reflect poor baseline recall rather than a general quality advantage of batching. In Filter→Map, F1 refers to the upstream filter's selection quality, not the semantic similarity of the final map output.

Pipeline	Workload	Per-tuple time (s) / calls	Fastest block sizes (k_1, k_2)	Best time (s) / calls	Speedup	F1 change
Filter → Join	100 reviews × 32, 10% pass	1.79 / 164	(1, 1)	1.79 / 164	1.00×	0.00
	25% pass	6.58 / 612	(1, 2)	4.93 / 356	1.33×	−0.10
	50% pass	10.64 / 996	(1, 4)	7.78 / 324	1.37×	−0.09
	75% pass	15.04 / 1,444	(1, 2)	10.40 / 772	1.45×	−0.13
	90% pass	20.19 / 1,956	(1, 4)	14.08 / 564	1.43×	−0.08
Filter → Map	50 reviews	2.30 / 69	(3, 1)	1.99 / 40	1.16×	+0.05
	250 reviews	4.45 / 368	(2, 1)	4.25 / 267	1.05×	+0.07

	500 reviews	7.71 / 735	(6, 1)	6.85 / 380	1.13×	+0.10
	1,000 reviews	15.86 / 1,477	(4, 4)	12.90 / 399	1.23×	+0.11
Map → Filter	200 reviews	7.08 / 401	(1, 2)	5.72 / 390	1.24×	+0.05
	500 reviews	11.93 / 1,001	(1, 1)	11.93 / 1,001	1.00×	0.00
Join → Filter	32 × 32 review pairs; 4 selectivities	10.88–11.46 / ~1,050	(2, 2) or (2, 3)	7.75–9.10 / 542–665	1.21–1.40 ×	+0.16 to +0.42*

Key Findings

1. Batching affects downstream workloads. At 90% filter selectivity, a blocked filter passed 78 rows instead of 58, increasing the join workload from 1,856 to 2,496 pairs.
2. The best isolated block sizes may not be optimal for pipelines. At 75% selectivity, Filter → Join performed best with (1, 2), achieving 1.45× speedup, even though batching the filter was faster in isolation.
3. Block utilization remains high. The scheduler accumulates tuples toward the configured block size, subject to the optional maximum block-token budget. In the nine inspected runs, the executed blocks were fully populated.
4. Batching can introduce additional retries. In one Map → Filter experiment, the map reran 282 of 500 rows because its estimated output-token budget was insufficient.
5. Small blocks generally work best for Join → Filter. Block sizes (2, 2) or (2, 3) achieved 1.21–1.40× speedups across four selectivities.
6. Pipeline stages have limited GPU overlap. Pipeline latency was approximately 0.98 times the sum of isolated stage latencies.

Conclusion

These results show that batch sizes must be selected jointly across a pipeline. Batching an operator can reduce its own execution time while increasing downstream work or retry costs. This motivates a pipeline-aware cost model that considers operator latency, block size, and the number of tuples passed between stages.

5.3 Patterns and summary

Case	Block?	Best k	Evidence	Why
Filter, short rows (up to ~500 tokens)	Yes	3–8	1.9–2.3× at 31 tokens; 1.33× at 146; 1.07× at 505	Repeated request overhead remains a meaningful share of latency
Filter, long rows (~1000+ tokens)	Usually no	1	0.84–1.02× across the ContractNLI selectivity runs	Reading the row dominates; blocking cannot avoid that work
Join, short pairs	Yes	2–4	~1.5× across review-pair match rates	Small classification answers amortize repeated instructions

Join, long pairs	No	1	0.97× at 139 tokens/pair, falling to 0.16× at ~1,600	Long pair prompts grow rapidly; tuple-wise requests benefit more from prefix reuse
Map, evaluated workloads	Usually no	1	At most 1.14×; commonly 0.4–0.9×; tuple-wise similarity is 0.3–0.55	One request must generate several substantive outputs sequentially
Very large blocks ($k \geq 16$)	Usually no	—	Slower than the best smaller tested block across the measured workloads	Longer output sequences and larger prompts outweigh fewer calls
Pipelines	Choose jointly	Per stage	Filter $k = 1$ + Join $k = 2$ achieved 1.45×; blocked filters can pass substantially more rows downstream	A stage's k changes downstream cardinality, retries, and generation cost
Overall	Depends	Depends	$k = 4$ ranges from 0.21× to 2.15×	Selection requires a model fitted to operator type, prompt/output lengths, pass rates, retries, and pipeline structure

6. Cost Model and Adaptive Block-Size Selection

No fixed block size is consistently best: with $k = 4$, measured speedup ranges from 0.21× to 2.15× across workloads. Kalypso therefore selects block sizes adaptively. Given a pipeline, the model estimates the end-to-end latency of each valid combination of per-stage block sizes and returns the fastest predicted configuration.

The model is fitted by weighted least squares using 173 single-operator measurements from 45 benchmark runs, and is composed and evaluated on 75 pipeline configurations. Short runs receive higher weight so that they are not overwhelmed by the largest workloads.

For each stage and candidate block size, the model estimates five effects:

1. Fixed setup cost: Each filter, join, and map has a small operator-specific fixed cost.
2. Prompt-reading cost: Larger blocks combine more rows into one prompt. This saves repeated instructions, but can make each prompt much longer.
3. Output-generation cost: A block returns one answer per input row, so larger blocks require longer responses. Map blocks are particularly expensive because their generated JSON outputs are long.
4. Call-group cost: vLLM can execute at most 256 requests concurrently. If a stage produces more requests, they run in multiple call groups; each additional group adds latency.
5. Pipeline effects: The block size of a filter or join can change its recall and false-positive rate, and therefore the number of rows sent to the next stage. Map stages may also issue one-row repair calls when a block output cannot be parsed or is incomplete.

The stage model combines these terms:

$$L_i = g_i + \alpha P_i + (\beta + \tau_{map}) W_i D_i + \gamma C_i D_i + \delta C_i (p_i/1000)^2$$

Where

g_i: Fixed operator startup cost

αP_i: Time spent reading the parts of the prompt that cannot be reused from the prefix cache

$(\beta + \tau_i) W_i D_i$: Time spent generating answers, accounting for how many call groups must run

$\gamma C_i D_i$: work required to generate all output tokens.

$\delta C_i (p_i/1000)^2$: Extra penalty when an individual prompt becomes long

C_i = ceil(N_i / k_i): LLM calls

W_i= ceil(C_i / *max_num_seqs*): number of call groups required to run all calls.

D_i : Generated tokens per call; blocks use approximately d0 + d1*k

p_i: Prompt tokens per call

τ_{map}: an additional per call-group decode-step cost for maps, reflecting that a call group completes only when its longest generated response finishes.

The model predicts stages in execution order. A join expands its input into candidate pairs, while a filter or join reduces its output according to the measured recall and false-positive rate at its selected block size. A map preserves row count but may add the predicted cost of repair calls.

The final pipeline estimate is: $L_{pipeline} = \lambda \sum_i L_i$,

Where $\lambda = 0.98$ accounts for small overlap between stages on one GPU.

Table below lists the fitted constants for our L40S / Qwen2.5-7B deployment; these are the values the block-size selector uses:

Fitted parameter	**Value: L40S + Qwen2.5-7B**	**Meaning**
Fixed cost, g: filter / join / map	−0.057 / −0.048 / 0.286 s	Regression intercept for each operator type
Prompt-reading cost, α	0.792 s per 10k computed prompt tokens	Time to process prompt tokens not served from the prefix cache
Call-group decode cost, β	24.8 ms per decode step	Base time for one output-generation step in a call group

Map tail cost, τ_{map}	11.0 ms per decode step	Extra cost for maps because a call group waits for its longest output
Output-work cost, γ	0.100 ms per generated token	Cost of generating all outputs across calls
Long-prompt attention cost, δ	1.77 ms per call per $(1k prompt tokens)^2$	Extra cost when an individual prompt is long
Block output length, $D = d0 + d1k$: filter / join / map	$-2.64 + 4.46k$ / $-2.61 + 4.57k$ / $63.32 + 14.74k$ tokens	Predicted generated tokens in one blocked call
Tuple-wise prompt overhead, q_{pt}: filter / join / map	133 / 157 / 128 tokens	Non-row prompt tokens in a $k = 1$ request
Short-filter recall: $k = 1 \rightarrow k \geq 2$	0.63 → 0.79	Fraction of truly matching rows passed onward
Short-filter FPR: $k = 1 \rightarrow k \geq 2$	0.013 → 0.030	Fraction of nonmatching rows incorrectly passed onward
Long-filter recall: $k = 1 \rightarrow k \geq 2$	0.70 → 0.45	Recall for long-row filters
Long-filter FPR: $k = 1 \rightarrow k \geq 2$	0.35 → 0.20	False-positive rate for long-row filters
Pipeline composition, λ	0.98	End-to-end latency is approximately 98% of the sum of predicted stage latencies
Map repair share, ρ	0–56% of map rows	Observed fraction requiring an additional one-row map call; varies by pipeline position and block choice

For each of the 173 single-operator measurements, the number of calls, call groups, prompt tokens and answer tokens are known from the run, so the stage formula is linear in the unknown constants (g, α, β, τ, γ, δ). We fit them by weighted least squares with measured latency as the target. Answer length ($d_0 + d_1 \cdot k$), prompt overhead (q_{pt}), recall and false-positive rates are measured directly, and λ is fitted on the 75 pipeline runs. The small negative intercepts for filter and join mean their fixed cost is effectively zero, not negative.

6.1 Block-size selection algorithm

Given a pipeline, the selector uses the fitted model to predict the end-to-end latency of every allowed combination of per-stage block sizes and returns the fastest one. A block size of k = 1 means the operator runs tuple-wise, so the same procedure decides both whether to batch and how large the blocks should be.

Algorithm : ChooseBlockSizes

Input: pipeline stages $op_1 \dots op_m$, input rows N, tokens per row t, right-side rows R for each join, selectivity s_i of each filter or join, fitted constants Θ. Output: a block size k_i for every stage ($k_i = 1$ means no batching).

1. **Bound the search.** For each stage j, the candidate set is $K_j = \{1, \dots, \min(k_ctx, k_meas, n_j)\}$, where k_ctx is the largest block whose prompt and answer fit in max_model_len, k_meas is the largest block size we measured for that operator and row-length class (short ≤ 512 tokens < long), and n_j is the number of rows reaching the stage.
2. **Optional accuracy guard.** If quality matters, remove every k whose measured F1 fell more than 0.02 below tuple-wise (for maps, ROUGE-1 below 0.8).
3. **Predict.** For every combination $(k_1, \dots, k_m)$ in $K_1 \times \dots \times K_m$, compute the pipeline latency with the model above, stage by stage, passing on the number of rows each filter or join is expected to keep at its chosen block size.
4. **Pick.** Let L* be the lowest predicted latency. Among all combinations predicted within 2% of L*, return the one with the smallest total block size, breaking ties by lower predicted latency. Preferring smaller blocks inside the error band protects accuracy and reduces the risk of long-output repairs.

The search is exhaustive, which is cheap in practice: a two-stage pipeline with at most 32 candidates per stage has about a thousand combinations, each a closed-form evaluation.

Worked example

Filter → Join on 100 reviews × 32, with 68% of reviews truly positive. A tuple-wise filter is expected to pass about 43 rows (0.63 × 68 + 0.013 × 32), while a blocked filter passes about 54 because its recall(0.79) is higher. Those 11 extra rows give the join roughly 350 more pairs. The model therefore keeps the filter tuple-wise and blocks only the join at k = 2, predicting 10.9 s. The measured time was 10.4 s (1.45× over tuple-wise), which was the fastest of all measured arms.

6.2 Algorithm vs static batching and per-tuple

Experiment	Per-tuple time (s)	Static k = 4	Static k = 16	Static k = 32	Algorithm's pick	Algorithm speedup	Best measured
Filter, 1500 short rows	14.6	2.15x	2.03x	1.56x	k= 6	2.23x	k=7, 2.26x
Join, 48×48 short rows	25.5	1.52x	1.33x	1.15x	k=2	1.52x	k=3, 1.54x
Filter → join (75%)	15.0	1.10x	0.94x	0.71x	filter k=1, join k=2	1.45x	same
Filter → map (250)	4.5	0.63x	0.26x	0.13x	per-tuple	1.00x	filter k=2, map k=1, 1.05x
Map → filter (200)	7.1	0.65x	0.23x	0.13x	per-tuple	1.00x	map k=1, filter k=2, 1.24x
Join → filter (join 50%, filter 50%)	11.4	1.28x	0.91x	0.62x	join k=10, filter k=2	1.18x	join k=2, filter k=2, 1.33x

Join → filter (62%, 10%)	10.9	1.30x	0.93x	0.64x	join 10, filter 2	1.22x	join k=2, filter k=, 1.40x
Join → filter (62%, 90%)	11.0	1.15x	0.83x	0.59x	join 10, filter 2	1.09x	join k=2, filter k=2, 1.21x
Join → filter (18%, 50%)	11.5	1.27x	0.90x	0.64x	join k=9, filter k=2	1.21x	join k=2, filter k=2, 1.29x
Filter, long rows (300)	39.7	0.93x	0.68x	–	per-tuple	1.00x	same
Join, long rows (40×8)	9.4	0.21x	–	–	per-tuple	1.00x	same
Filter, 146 tokens	7.7	1.31x	1.06x	0.83x	k=2	1.27x	k=3, 1.33x
Filter, 267 tokens	11.4	1.15x	0.97x	0.79x	k=2	1.12x	k=3, 1.15x
Filter, 505 tokens	20.1	1.06x	0.88x	0.74x	k=2	1.05x	k=3, 1.07x
Filter, 937 tokens	33.0	0.98x	0.76x	0.59x	k=2	1.00x	same
Join, 139 tokens	3.5	0.82x	0.58x	0.43x	k=2	0.97x	per-tuple
Join, 261 tokens	4.1	0.55x	0.41x	0.32x	per-tuple	1.00x	same
Join, 501 tokens	4.9	0.38x	0.27x	0.22x	per-tuple	1.00x	same
Join, 942 tokens	6.5	0.27x	–	–	per-tuple	1.00x	same
Map, 125 tokens	10.1	0.94x	–	–	per-tuple	1.00x	k=2, 1.03x
Map, 244 tokens	13.1	0.80x	–	–	per-tuple	1.00x	same
Map, 485 tokens	23.1	0.78x	–	–	per-tuple	1.00x	same
Map, 916 tokens	41.9	0.71x	–	–	per-tuple	1.00x	same
Average over all experiments		0.85x	0.71x	0.52x		1.12x	1.16x

Model accuracy. Prediction error is the gap between predicted and measured latency.

Evaluated on	Mean error	Max error
Single operators, fitted	4.3%	36%
Single operators, one workload left out	4.8%	61%
Pipelines, fitted	13.2%	87%
23 held-out experiments, the configuration the algorithm picked	6.5%	25%

The model is least accurate on pipelines, mainly because it must also predict how many rows each stage passes on and how many map rows need repair calls. It is most accurate where it matters for the decision: on the configuration it actually chooses.

Inference

A fixed block size is unreliable, while the adaptive algorithm is usually close to the best tested choice.

Finding	Evidence
No static size is safe	Static $k = 4$ averages only **0.85×**: 15% slower than tuple-wise overall. Static $k = 16$ and $k = 32$ are worse: **0.71×** and **0.52×**.
$k = 4$ can be excellent or disastrous	It gives **2.15×** on the short filter, but only **0.21×** on the long join.
Bigger blocks are consistently risky	$k = 16$ and $k = 32$ lose performance in almost every pipeline and long-input case.
Short filters and short joins benefit from small blocks	The algorithm selects filter $k = 6$ for 1,500 short rows and join $k = 2$ for short review pairs. Both are near the best measured choice.
Long rows should not be blocked	For long filters and joins, the algorithm selects tuple-wise execution, which is the measured best choice.
Maps generally should not be blocked	The algorithm selects tuple-wise map execution in almost all cases. It misses only a tiny gain at 125-token maps: 1.00× versus the oracle's 1.03×.
Pipeline stages must be chosen jointly	In Filter → Join, the algorithm selects filter $k = 1$, join $k = 2$, obtaining **1.45×**, exactly the best measured configuration.
The model is conservative when uncertain	In Filter → Map and Map → Filter, it selects per-tuple execution. It misses modest possible gains of 5% and 24%, but avoids the severe losses caused by static batching.
The main miss is Join → Filter	The algorithm chooses a large join block ($k \approx 9$–10), while the fastest run was with join $k = 2$. It still speeds up execution, but not as much as the best run result..